Data Engineer — 45-Minute Mock Interview

Timed, mixed questions with model answers — simulate the real thing

warm-up · concepts · SQL/coding · system design · your questions · scoring rubric

Simulate a real interview. Set a 45-minute timer, answer each question **out loud** in your own words, then read the model answer. Sections are timed like a typical DE loop: warm-up → concepts → SQL/coding → system design → your questions.

> How to score yourself: did you (1) answer the question, (2) give a concrete example, and (3) mention the *why* (performance, cost, reliability, security)? That trio = a strong answer.

0:00–0:05 — Warm-up & behavioral

Q1. Tell me about yourself. *Model:* "I'm a data engineer with ~6 years building pipelines, lakes, and warehouses on AWS and Azure, plus a strong earlier background in Java/backend. Day to day I work with Glue/Spark, S3, Redshift/Snowflake, and dbt — building batch and streaming ELT. I care about clean data models, performance, cost, and data quality. Most of my work has been for banks and financial clients, so I've done a lot of production support. Most recently I built an S3-to-Snowflake ELT flow with Glue and dbt, fully deployed with Terraform." *(30–60s, then stop.)*

Q2. Tell me about a hard problem you solved. *(STAR) Model:* "**S**: An audit job over millions of account records kept failing with out-of-memory. **T**: Make it stable and fast. **A**: I took a heap/plan look, found a single stage loading a huge result set and a big shuffle; I added a broadcast for the small dimension, filtered earlier, repartitioned, and right-sized the cluster. **R**: OOMs gone, runtime cut sharply, and I added a monitoring alert so it couldn't silently regress." *(End with the metric and the prevention.)*

0:05–0:20 — Technical concepts (rapid mix)

Q3. ETL vs ELT — which do you use and why? *Model:* "ELT for lake/warehouse work — land raw first, transform in-place with Spark/SQL/dbt. It scales, keeps raw for reprocessing, and lets the warehouse do the heavy lifting. I use ETL when I must cleanse or mask data *before* it lands, e.g., PII."

Q4. Walk me through what happens when you submit a Spark job. *Model:* "`spark-submit` starts the **driver**, which asks the **cluster manager** for **executors** on worker nodes. The driver builds a **DAG** from my lazy transformations and Catalyst optimizes it. When an **action** fires, the **DAG scheduler** splits the job into **stages** at shuffle boundaries; the **task scheduler** sends one **task per partition** to executors that run them in parallel. Narrow ops stay on the partition; wide ops like groupBy/join cause a **shuffle** across the network. For a big job I'd broadcast small tables, watch for skew, and coalesce before writing."

Q5. How do you make a pipeline incremental and idempotent? *Model:* "Incremental: process only new/changed data using a **watermark**, **CDC**, or **job bookmarks**, and **MERGE/upsert** for updates. Idempotent: overwrite-by-partition or MERGE on a key so re-running doesn't double-count. I keep raw immutable so I can always reprocess."

Q6. Parquet + partitioning — why do they matter? *Model:* "Parquet is columnar and compressed, so queries scan only needed columns. Partitioning by a filter column (usually date) lets the engine skip irrelevant folders — partition pruning. Together they cut scan time and cost dramatically. I also avoid the small-files problem by compacting/coalescing."

Q7. What's Delta Lake / Iceberg and why use them? *Model:* "Open table formats on Parquet that add **ACID transactions, upserts/MERGE, time travel, and schema evolution** on object storage — so a lake behaves like a reliable warehouse. Delta is Databricks-centric; Iceberg is engine-neutral. They're how you do updates/deletes on a lake that's otherwise append-only."

Q8. How do you secure data in a pipeline? *(the differentiator — nail this) Model:* "Encryption everywhere — **KMS** at rest, **TLS** in transit. **Least-privilege IAM** roles per service, temporary credentials, no wildcards. Secrets in **Secrets Manager**, never in code, rotated. For PII: classify, **mask/tokenize**, column/row-level security, and audit access with CloudTrail. In code: **parameterized queries** to prevent SQL injection, input validation, and dependency scanning."

Q9. Redshift DISTKEY vs SORTKEY? *Model:* "DISTKEY controls *where* rows live — put it on the common join column so joined rows sit on the same slice and avoid a shuffle. SORTKEY controls on-disk *order* — usually a date, so range filters skip blocks via zone maps. Small dimensions I set to DISTSTYLE ALL."

Q10. Dimensional vs normalized modeling, and what's SCD Type 2? *Model:* "Normalized (3NF) minimizes redundancy — good for OLTP/integration. Dimensional (star: facts + denormalized dimensions) is for fast analytics. SCD Type 2 tracks history in a dimension by adding a new row per change with effective/expiry dates and a current flag, usually with a surrogate key."

0:20–0:30 — SQL / coding

Q11. Write SQL: latest record per order (dedup). *Model:*

```
SELECT order_id, customer_id, amount, status
FROM (SELECT o.*, ROW_NUMBER() OVER (PARTITION BY order_id ORDER BY order_ts DESC) rn
      FROM orders o) t
WHERE rn = 1;
```

"I partition by the key, order by timestamp descending, and keep row_number = 1 — the standard CDC/upsert dedup."

Q12. Write SQL: top 3 products by revenue per category. *Model:*

```
SELECT * FROM (
  SELECT p.category, p.product_name, SUM(o.amount) rev,
         ROW_NUMBER() OVER (PARTITION BY p.category ORDER BY SUM(o.amount) DESC) rn
  FROM orders o JOIN products p ON p.product_id=o.product_id
  GROUP BY p.category, p.product_name) t
WHERE rn <= 3;
```

Q13. PySpark: clean orders and write partitioned Parquet. *Model:*

```
from pyspark.sql import functions as F, Window
w = Window.partitionBy("order_id").orderBy(F.col("order_ts").desc())
clean = (orders
   .filter((F.col("status")=="completed") & (F.col("amount")>0))
   .withColumn("amount", F.col("amount").cast("double"))
   .withColumn("rn", F.row_number().over(w)).filter("rn=1").drop("rn"))
clean.withColumn("dt", F.to_date("order_ts")) \
     .write.mode("overwrite").partitionBy("dt").parquet("s3://.../curated/orders/")
```

(Say aloud: "filter early, cast, dedup with a window, partition by date on write.")

0:30–0:42 — System design / scenario

Q14. Design a pipeline: daily order files land in S3; the business wants a next-morning revenue dashboard and data for an ML model. *Model (walk through it):*

1. **Ingest / land** raw files in `s3://.../raw/orders/dt=YYYY-MM-DD/`, immutable, partitioned.
2. **Transform** with a **bookmarked Glue (PySpark)** job (or Databricks): clean, dedupe, cast, join dimensions (broadcast the small ones), write **Parquet** to `curated/`, partitioned by date.
3. **Model** a **star schema** (fact_orders + dim_customer/product/date), incremental for the big fact via **MERGE** (Delta/Iceberg).
4. **Catalog** with a Glue Crawler (or explicit table + partition projection).
5. **Serve** BI on a dedicated warehouse (Redshift/Snowflake) via COPY, on its own compute so it doesn't contend with ETL; **feature tables** for the ML model.
6. **Orchestrate** with Airflow/Step Functions — ingest → transform → crawl → load → quality — with retries/alerts, event-triggered on new files (EventBridge).
7. **Data quality:** dbt/GE tests (null/unique/range/freshness) that **fail** the run on bad data.
8. **Ops:** Terraform + CI/CD, CloudWatch alarms, cost control via partitioning/Parquet/right-sizing. *(Then mention idempotency: overwrite the day's partition on reruns; and backfills: reprocess a date range.)*

Q15. Follow-up: the dashboard got slow as data grew to billions of rows — what do you do? *Model:* "Check the query plan. On Redshift I'd set SORTKEY on the date the dashboard filters by, DISTKEY on the join column, add a **materialized view** for the daily aggregate, and enable concurrency scaling for peak hours. On the lake side, ensure Parquet + partitioning so Athena scans less. If it's Spark, look for skew and a missing broadcast."

Q16. Follow-up: how do you handle late-arriving data? *Model:* "Partition by event date, and when late data lands, reprocess just the affected partition (idempotent overwrite) or MERGE the changes in — so history stays correct without a full reload."

0:42–0:45 — Your questions (ask 2–3)

Strong questions to ask *them*:

- "What's the current stack — Snowflake, Redshift, Databricks, or a mix — and any migration in flight?"
- "Batch, streaming, or both, and what are the SLA/latency expectations?"
- "How mature is your orchestration, CI/CD, and data-quality tooling?"
- "What's the biggest data-engineering pain point on the team right now?"

Scoring rubric (rate each answer 1–3)

	1 (weak)	2 (ok)	3 (strong)
Answered the question	vague/rambling	mostly	direct + structured
Concrete example	none	generic	specific from experience
The "why"	missing	mentioned	tied to perf/cost/reliability/security
Communication	jargon-heavy	clear	clear + confident, right length

Aim for mostly 3s. After the mock, note the 2–3 questions where you scored lowest and drill those from the JD-specific guides and the rapid-fire drill.